

XPCOM Type Library File Format

Version 1.0, Draft 1

Last updated: 04/02/2026 22:09:04

Author: [Scott Furman <fur@netscape.com>](mailto:fur@netscape.com)

Latest version:

http://mozilla.org/scriptable/typelib_file.html

This version:

http://mozilla.org/scriptable/typelib_file_v1_d1.html

Introduction

XPCOM type libraries, or "typelibs", are binary interface description files generated by the [XPIDL](#) compiler. Type libraries enumerate the methods of one or more interfaces, including detailed type information for each method parameter. Typelibs might be more aptly named "interface libraries", but Microsoft has already established a precedent with their naming scheme and we'll stick with it to avoid developer confusion.

Goals

- Represent all type information that is required for [XPCOMConnect](#) to enable access to COM objects from JS and vice-versa
- Map an IID to an interface description quickly
- Allow for efficiency when interfaces are sparsely used, i.e. without requiring the entire typelib to be parsed or retained in memory.
- Use a compact file format, so as to accommodate hundreds of interfaces and thousands of methods.
- Allow for extensibility, so as to accommodate future additions to XPIDL

Non-goals

- Ability to reconstruct XPIDL source from type library
- Human-readable format
- File-level compatibility with MS typelibs

Notation

The syntax used in this document to specify the layout of file data appears similar to C structs. Unlike C structs, however, data members are not subject to alignment restrictions. Another difference from C structs is the use of pointer notation to represent 32-bit file offsets. For example,

```
uint16 *ByteOffsetTo16BitValue
```

specifies a 32-bit field that contains the offset, in bytes to an array of one or more 16-bit values. Unless otherwise noted, all file offsets are byte offsets from the beginning of the [data pool](#) and are 32-bit signed quantities. The first byte of the data pool is at offset 1, so as to allow offset 0 to be used as a special

indicator. By adding in an appropriate constant, these offsets are appropriate as arguments to `seek()`.

Record fields with type `boolean` occupy one bit, not one byte. A value of 1 represents true and a value of 0 represents false.

All integer fields with multibyte precision are stored in little-endian order, e.g. for a `uint16` field, the low-order byte is stored in the file followed by the high-order byte.

Filename Suffix

The standard suffix for XPCOM type libraries is `.xpt`. (*Do we need to define a standard four-character Mac signature/creator ?*)

File Header

Every XPCOM typelib file begins with a header:

```
TypeLibHeader {
    char    magic[16];
    uint8   major_version;
    uint8   minor_version;
    uint16  num_interfaces;
    uint32  file_length;
    InterfaceDirectoryEntry* interface_directory;
    uint8*  data_pool;
}
```

magic

The first 16 bytes of the file always contain the following values:

```
(hex) 58 50 43 4f 4d 0a 54 79 70 65 4c 69 62 0d 0a 1a
(C notation) X P C O M \n T y p e L i b \r \n \032
```

This signature both identifies the file as an XPCOM typelib file and provides for immediate detection of common file-transfer problems, i.e. treatment of a binary file as if it was a text file. The CR-LF sequence catches file transformations that alter newline sequences. The control-Z character stops file display under MS-DOS. The linefeed in the sixth character checks for the inverse of the CR-LF translation problem. (*A nod to the PNG folks for the inspiration behind using these special characters in the header.*)

major_version, minor_version

These are the major and minor version numbers of the typelib file format. For this specification `major_version` is 0x01 and `minor_version` is 0x00. TypeLib files that share the same major version but have different minor versions are compatible. Changes to the major version represent typelib file formats that are not backward-compatible with parsers designed only to read earlier major versions. If a typelib file is encountered with a major version for which support is not available, the rest of the file should not be parsed.

num_interfaces

This indicates the number of [InterfaceDirectoryEntry](#) records that are at the offset indicated by the `interface_directory` field.

interface_directory

This field specifies a zero-relative byte offset from the beginning of the file. It identifies the start of an array of `InterfaceDirectoryEntry` records. If `num_interfaces` is zero, then this field should also be zero. The value of this field should be a multiple of 4, i.e. the interface directory must be aligned on a 4-byte boundary. (This is to guarantee aligned access if the typelib file is mmap'ed into memory.)

file_length

Total length of the typelib file, in bytes. This value can be compared to the length of the file reported by the OS so as to detect file truncation.

data_pool

The data pool is a heap-like storage area that is the container for most kinds of typelib data including, but not limited to [InterfaceDescriptor](#), [MethodDescriptor](#), [ParamDescriptor](#), and [TypeDescriptor](#) records. Note that, unlike most file offsets in a typelib, the value of `data_pool` is relative to the beginning of the file.

InterfaceDirectoryEntry

A contiguous array of fixed-size `InterfaceDirectoryEntry` records begins at the byte offset identified by the [interface_directory](#) field in the [file header](#). The array is used to quickly locate an interface description using its IID. No interface should appear more than once in the array.

```
InterfaceDirectoryEntry {  
    uint128      iid;  
    InterfaceDescriptor* interface_descriptor;  
}
```

iid

The `iid` field contains a 128-bit value representing the interface ID. The `iid` is created from an IID by concatenating the individual bytes of an IID in a particular order. For example, this IID:

{00112233-4455-6677-8899-aabbccddeeff}

is converted to the 128-bit value

0xffeeddccbbaa88996677445500112233

Note that the byte storage order corresponds to the layout of the `nsIID` C-struct on a little-endian architecture.

All `InterfaceDirectoryEntry` objects must appear sorted in increasing order of `iid`, so as to facilitate a binary search of the array.

interface_descriptor

This is a byte offset from the beginning of the file to the corresponding `InterfaceDescriptor` object.

InterfaceDescriptor

An InterfaceDescriptor is a variable-size record used to describe a single XPCOM interface, including all of its methods:

```
InterfaceDescriptor {
    String          name;
    InterfaceDescriptor* parent\_interface;
    uint16         num\_methods;
    MethodDescriptor method\_descriptors[num_methods];
}
```

name

The human-readable name of this interface, e.g. "nsISupports", stored using the [String](#) record format.

parent_interface

As a space-saving measure, an interface's methods can be specified by composing the methods of an interface from which it is derived with additional methods it defines. In this case, the `method_descriptors` array does not list any methods that the interface inherits from its parent and the `parent_interface` field contains a byte offset to the InterfaceDescriptor of its parent interface. Note: the XPIDL compiler is not required to compress interface descriptions in this manner - it's purely a space optimization.

num_methods

The number of methods in the `method_descriptors` array.

method_descriptors

This is a byte offset from the beginning of the data pool to an array of MethodDescriptor objects. The length of the array is determined by the `num_methods` field.

MethodDescriptor

A MethodDescriptor is a variable-size record used to describe a single interface method:

```
MethodDescriptor {
    boolean    is\_getter;
    boolean    is\_setter;
    uint6      reserved;
    String*    name;
    uint8      num\_args;
    ParamDescriptor params[num_args];
    ParamDescriptor result;
}
```

is_getter

This field is used to allow interface methods to act as property getters for object-oriented languages such as JavaScript. For example, if there was an XPCOM method named

"get_Banjo", you could access a "Banjo" property on an interface like this: 'myInterface.Banjo'. If `is_getter` is true, then the "get_" prefix should be stripped off the method's name by the XPIDL compiler.

is_setter

This field is used to allow interface methods to act as property setters for object-oriented languages such as JavaScript. For example, if there was an XPCOM method named "set_Banjo", you could assign to a "Banjo" property on an interface like this: 'myInterface.Banjo = 3'. If `is_setter` is true, then the "set_" prefix should be stripped off the method's name by the XPIDL compiler.

name

The human-readable name of this method, e.g. "getWindow", stored in the [String](#) record format.

num_args

The number of arguments that the method consumes. Also, the number of elements in the `params` array.

params

This is a byte offset from the beginning of the data pool to an array of [ParamDescriptor](#) objects. The length of the array is determined by the `num_args` field.

result

This is a byte offset from the beginning of the data pool to a single [ParamDescriptor](#) object that identifies the type of the method return value.

ParamDescriptor

A `ParamDescriptor` is a variable-size record used to describe either a single argument to a method or a method's result:

```
ParamDescriptor {  
    boolean    in;  
    boolean    out;  
    uint64     reserved;  
    TypeDescriptor type;  
}
```

in

If `in` is true, it indicates that the parameter is to be passed from caller to callee.

out

If `out` is true, it indicates that the parameter is to be passed from callee to caller. Out parameters must have pointer type. It is possible for a parameter to have both `out` and `in` bits set.

reserved

A 6-bit field reserved for future use.

type

The type of the method parameter.

TypeDescriptor

A TypeDescriptor is a variable-size record used to identify the type of a method argument or return value. There are many XPCOM types that need to be represented in the typelib:

- scalar types, e.g. uint16 or boolean
- pointer to a scalar type, e.g. float*
- generic opaque pointer, i.e. void*
- void (only allowed for method return type)
- interface pointer, or a pointer to an interface pointer
- iid_is - that is, an interface pointer with type determined at runtime, or a pointer to such an interface pointer
- pointer to an nsIID structure (to act as the parameter to an iid_is)

[Editor: This specification does not yet cover pointers to unions, structs or arrays.]

To efficiently describe all the type categories listed above, there are several different variants of TypeDescriptor records:

```
union TypeDescriptor {
    SimpleTypeDescriptor;
    InterfaceTypeDescriptor;
    InterfaceIsTypeDescriptor;
}
```

The first byte of all these TypeDescriptor variants has the identical layout:

```
TypeDescriptorPrefix {
    boolean  is\_pointer;
    boolean  is\_unique\_pointer;
    boolean  is\_reference;
    uint5    tag;
}
```

is_pointer

This field is true only when representing C pointer/reference types.

is_unique_pointer

This field cannot have a value of true unless is_pointer is also true. The unique_pointer field indicates if the parameter value can be aliased to another parameter value. If unique_pointer is true, it must not be possible to reach the memory pointed at by this argument value from any other argument to the method.

is_reference

This field cannot have a value of true unless `is_pointer` is also true. This field is true if the parameter is a reference, which is to say, it's a pointer that can't have a value of NULL.

tag

The tag field indicates which of the [variant TypeDescriptor records](#) is being used, and hence the way any remaining fields should be parsed.

Value in tag field	TypeDescriptor variant to use
0..15	SimpleTypeDescriptor
16	InterfaceTypeDescriptor
17	InterfaceIsTypeDescriptor
18..31	reserved

SimpleTypeDescriptor

The one-byte SimpleTypeDescriptor is a kind of [TypeDescriptor](#) used to represent scalar types, pointers to scalar types, the void type, the void* type and, as a special case, the nsIID* type:

```
SimpleTypeDescriptor {  
    boolean is\_pointer;  
    boolean is\_unique\_pointer;  
    boolean is\_reference;  
    uint5 tag;  
}
```

is_pointer, tag

The `is_pointer` field is true only when representing C pointer/reference types. The following table indicates the types that can be represented by combining different values for these two fields:

SimpleTypeDescriptor		
pointer	tag	Type Represented
false	0	int8
false	1	int16
false	2	int32
false	3	int64
false	4	uint8
false	5	uint16
false	6	uint32
false	7	uint64
false	8	float
false	9	double
false	10	boolean (8-bit value)

false	11	char (8-bit character)
false	12	wchar_t (16-bit character)
false	13	void
false	14	reserved
false	15	reserved
true	0	int8*
true	1	int16*
true	2	int32*
true	3	int64*
true	4	uint8*
true	5	uint16*
true	6	uint32*
true	7	uint64*
true	8	float*
true	9	double*
true	10	boolean* (8-bit value)
true	11	char* (8-bit character)
true	12	wchar_t* (16-bit character)
true	13	void* (generic opaque pointer)
true	14	nsIID*
true	15	string

InterfaceTypeDescriptor

An `InterfaceTypeDescriptor` is used to represent either a pointer to an interface type or a pointer to a pointer to an interface type, e.g. `nsISupports*` or `nsISupports**`:

```
InterfaceTypeDescriptor {
    boolean    is_pointer;
    boolean    is_unique_pointer;
    boolean    is_reference;
    uint5      tag;
    InterfaceDescriptor* interface;
}
```

is_pointer

When this field is false, the represented type is an interface pointer. When `is_pointer` is true, the represented type is a pointer to an interface pointer.

tag

The tag field must have the decimal value 16.

InterfaceIsTypeDescriptor

An `InterfaceIsTypeDescriptor` describes an interface pointer type. It is similar to an [InterfaceTypeDescriptor](#) except that the type of the interface pointer is specified at runtime by the value of another argument, rather than being specified by the typelib.

```
InterfaceIsTypeDescriptor {  
    boolean is\_pointer;  
    boolean is\_unique\_pointer;  
    boolean is\_reference;  
    uint5 tag;  
    uint8 arg\_num;  
}
```

tag

The tag field must have the decimal value 17.

arg_num

The index of the method argument that describes the type of the interface pointer. The specified method argument must have type `nsIID*`.

String

String records are used to represent variable-length, human-readable strings:

```
String {  
    uint16 length;  
    char bytes[];  
}
```

length

The length of the string, in characters (not bytes).

bytes

Unicode string encoded in UTF-8 format, with no null-termination. The length of the bytes array, measured in Unicode characters (not bytes), is reported by the `length` field.